

MHEL FINANCE

Backend-First Development Roadmap

Automated personal expense tracking designed to grow into a multi-user product

Roadmap version	1.0
Date	24 August 2026
Primary repo	mhel-finance-api
Runtime	Cloudflare Workers

Architecture decision: Use separate repositories for independently deployable clients and API, while keeping the Hono backend itself as one modular application. Repository separation improves release and team independence; it does not require microservices.

Basis: project requirements established in this conversation plus the uploaded Backend Architecture Specification (Hono / Drizzle / Neon / Better Auth / Zod / Cloudflare Workers).

1. Executive Direction

Product goal. Build an exception-driven expense system: transactions are captured automatically, normalized into one ledger, categorized by deterministic rules first, and surfaced to you only when confidence is low.

Backend goal. Create a reliable financial core that is simple enough for personal use now, but already carries the ownership, API, migration, security, and contract boundaries needed for future users.

North-star workflow: Capture -> persist raw observation -> normalize -> deduplicate -> resolve account/merchant -> classify -> detect transfers -> create/update canonical transaction -> review only exceptions -> reconcile with statements.

Success criteria for the backend MVP

- Flutter can submit Android notification events safely and idempotently.
- Gmail ingestion can add bank/e-wallet notification events without manual encoding.
- One real-world purchase observed through multiple sources becomes one canonical transaction.
- Known merchants and user rules classify automatically; Workers AI is used only as a fallback.
- Transfers and credit-card payments do not inflate spending.
- Low-confidence items enter a review queue and corrections improve future automation.
- The API is documented by OpenAPI and can generate clients independently in Flutter and Nuxt.
- Every finance row is owned by a user from day one, even while the product has only one user.

Technology baseline

Concern	Decision	Reason
HTTP/API	Hono on Cloudflare Workers	Small, fast edge runtime and already familiar.
Database	Neon PostgreSQL + Drizzle ORM	Relational integrity, SQL visibility, schema/migration discipline.
Validation/OpenAPI	Zod + @hono/zod-openapi + drizzle-zod	Schema-driven runtime validation and cross-repo API contract.
Authentication	Better Auth	Cookie sessions for Nuxt; bearer-compatible auth for Flutter.
AI	Cloudflare Workers AI	Fallback merchant/category inference behind an injected adapter.
Files - later	Cloudflare R2	Original statements and import artifacts without bloating Postgres.
Background work	Cloudflare Cron + waitUntil initially	Minimal moving parts; queue can be added when throughput requires it.

2. Repository Strategy

Important correction: Separate repositories are a deployment and ownership boundary, not a traffic-scaling mechanism. User-scale comes from stateless Workers, database design, idempotency, indexing, and resilient processing. Do not split backend modules into separate services until a real boundary appears.

```
mhel-finance-api      # Hono + Neon + Better Auth + Workers AI + automation
mhel-finance-mobile  # Flutter / Android ingestion + daily review UX
mhel-finance-web     # Nuxt / desktop analytics + configuration
```

Rules for a multi-repo setup

- The API repo owns the database schema, migrations, OpenAPI document, and backend business rules.
- The mobile and web repos never duplicate backend DTO definitions by hand.
- The API publishes a versioned OpenAPI artifact. Flutter generates a Dart client; Nuxt generates a TypeScript client.
- No shared source-code package is required initially. The OpenAPI contract is the integration seam.
- Infrastructure directly coupled to the API (wrangler config, Worker bindings, migrations) stays in the API repo.
- A new backend repository is created only when an independently deployable workload has a concrete scaling, security, or team-ownership reason.

Future repo split triggers

Possible repo	Create only when...
mhel-finance-ingestion-worker	Ingestion volume or provider polling needs independent scaling/deploy cadence.
mhel-finance-reporting-worker	Long-running reporting/exports exceed normal API Worker execution patterns.
mhel-finance-platform	Shared infrastructure becomes substantial enough to justify a dedicated owner/release lifecycle.

3. Hono Architecture Rules

The API repo should follow the uploaded architecture guide, not invent a second style. The most important retained rules are explicit dependency injection, thin transport handlers, vertical slices, schema-driven contracts, and business logic that can run without HTTP. These are described in the guide on pages 3-7 and 10-19.

Layer	May do	Must not do
Route	Validate transport input, call one service method, serialize output.	SQL, Drizzle queries, domain decision logic.
Service	Business rules, authorization at resource level, transaction boundaries.	Hono Context, c.req, Response, HTTP status codes.
Repository	Drizzle/SQL queries and persistence translation.	Business rules, HTTP, other modules repositories.
DB schema	Tables, indexes, constraints, relations.	Application logic.

```

src/
├── index.ts
├── container.ts
├── env.ts
├── core/
│   ├── errors.ts
│   ├── http/
│   ├── middleware/
│   ├── pagination/
│   └── logger.ts
├── db/
│   ├── client.ts
│   └── schema/
├── auth/
├── triggers/
├── modules/
│   ├── account/
│   ├── ingestion/
│   ├── transaction/
│   ├── merchant/
│   ├── categorization/
│   ├── transfer/
│   ├── review/
│   ├── budget/
│   ├── integration/
│   ├── statement/
│   └── report/
└── # thin cron/scheduled transport adapters

```

Composition root: container.ts remains the single readable object graph: env -> database -> repositories -> services -> route/trigger adapters. Use factory functions and explicit dependency passing; do not add a DI container.

4. Canonical Financial Data Model

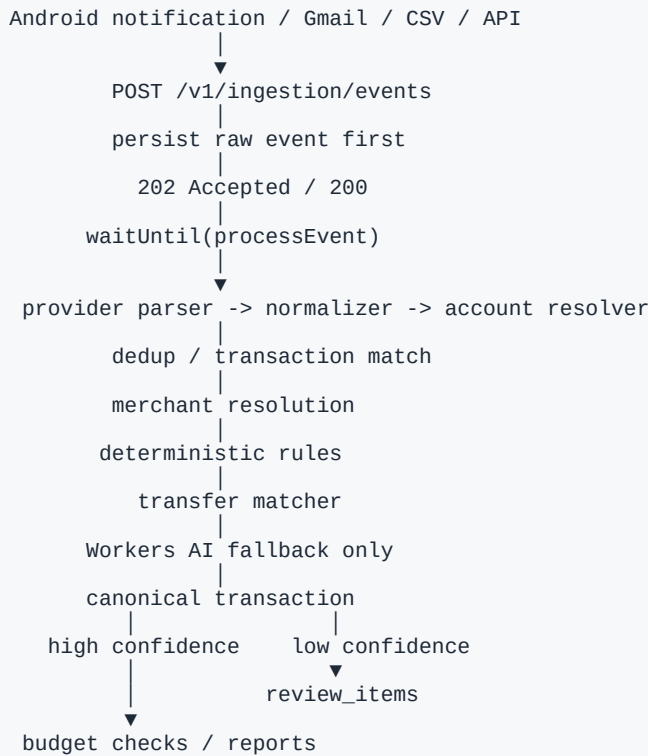
Critical modeling decision: an email, Android notification, API event, and statement row can all describe the same transaction. Keep raw observations separate from the canonical ledger.

Table / aggregate	Purpose	Day-1 invariant
financial_accounts	Bank, card, e-wallet, cash accounts.	Always owned by user_id; account type and currency required.
raw_financial_events	Immutable-ish source observations from Android, Gmail, imports, APIs.	Unique (user_id, source_type, external_id); processing status retained.
transactions	Canonical ledger entries.	Signed amount_minor bigint; currency; account; transaction time; type/status.
transaction_sources	Links one transaction to one or many raw observations.	Prevents duplicate spending when multiple sources observe one event.
merchants / merchant_aliases	Normalized merchant identity plus learned raw-name aliases.	Alias matching is user-scoped where needed.
categories	User-visible spending categories.	System defaults may be copied/overridden per user later.
classification_rules	Deterministic automation rules.	Rules are evaluated before AI.
transfer_groups	Links both sides of internal money movement.	Transfer impact on expense totals = zero.
review_items	Explicit exception queue.	Every ambiguous decision has reason + confidence/evidence.
budgets	Category/context limits by period.	Independent of transaction ingestion.
integration_connections	Gmail/provider tokens and sync cursors.	Tokens encrypted before storing.
statements / statement_rows	Imported reconciliation sources.	Original file retained in R2; parsed rows reproducible.

Money and ownership rules

- Never use floating-point money. Store minor units as bigint (for example PHP 124900 = PHP 1,249.00).
- Store currency separately using a 3-character currency code.
- Use signed amounts relative to the account: outflow negative, inflow positive.
- Store raw merchant text separately from normalized merchant_id.
- Put user_id on every user-owned aggregate now. This is cheaper than retrofitting tenancy later.
- Include user_id in unique constraints and important indexes so one user cannot collide with another.
- Keep event payload retention deliberate; financial notification text is sensitive data.

5. Event Processing and Automation Model



Reliability pattern for v1: Persist the raw event before doing expensive work. Use `c.executionCtx.waitUntil()` for fast follow-up processing, and run a Cron retry sweep for events left pending/failed. This avoids introducing a queue before it is necessary while still making failed work recoverable.

When to add Cloudflare Queues

- Processing latency must be independent from API traffic.
- Provider polling or mobile events create sustained backlogs.
- You need explicit retry/dead-letter behavior at higher volume.
- A processing worker must scale or deploy independently from the API.

Do not use KV as the financial idempotency source of truth. Use PostgreSQL unique constraints for event identity and transaction/source linkage. KV can still be used for generic rate-limit or short-lived response caching.

6. Workers AI Integration

AI is an inference adapter, not the ledger authority. The categorization service decides whether AI is needed, validates the model output, and applies policy. The model never writes directly to Postgres.

- 1. Exact and deterministic matching** - Known source parser, account mapping, exact merchant alias, explicit user rule.
- 2. Heuristic matching** - Merchant normalization, transfer candidate scoring, known text patterns.
- 3. Workers AI fallback** - Only when deterministic evidence cannot classify with enough confidence.
- 4. Review** - Human correction for ambiguous items; correction teaches a safe rule/alias for future events.

AI adapter contract

```
interface TransactionClassifier {
  classify(input: {
    merchantRaw: string
    description?: string
    amountMinor: bigint
    currency: string
    availableCategories: CategoryRef[]
  }): Promise<ClassificationSuggestion>
}
```

Policy	Decision
Data minimization	Send only transaction text needed for classification; never full auth tokens or full account identifiers.
Structured output	Require strict JSON fields: merchant candidate, category, context, rationale code, model score.
Validation	Parse output with Zod; reject unknown category IDs and malformed output.
Confidence	Do not blindly trust model self-confidence. Compute system confidence from evidence type.
Learning	One correction may create an exact alias. Avoid broad wildcard rules until repeated evidence exists.
Audit	Store model identifier, schema/prompt version, result, and processing outcome for debugging/evaluation.

7. Authentication and Multi-User Readiness

Authentication is needed now: Even for one user, the API holds highly sensitive financial data and is consumed by mobile/web clients. Use Better Auth from the first deploy rather than adding auth after data exists.

Client	Auth mode	Notes
Flutter	Bearer-compatible session token	Use Better Auth bearer support; secure token storage on device.
Nuxt	Secure HTTP-only cookie session	Standard browser session; CORS allowlist only.
Automations	User-owned integration connection or scoped service credential	Do not hard-code a universal backdoor token.

Personal mode now, product mode later

- Create the initial account and disable open registration with configuration after bootstrap.
- Every service method that reads user data receives the actor/principal or explicit userId from a trusted caller.
- Repository queries for finance data are scoped by user_id.
- Resource ownership checks live in services, consistent with the uploaded architecture guide.
- Defer subscriptions, entitlements, organizations, teams, and billing until monetization is actually being built.
- Consider Postgres RLS as defense-in-depth when multi-user beta begins; do not add it merely for single-user v1.

8. Backend API Surface - Build Order

Module	Initial endpoints / triggers	Milestone
system	GET /health, GET /openapi.json, docs	M1
auth	/api/auth/* via Better Auth	M2
accounts	GET/POST/PATCH /v1/accounts	M3
transactions	GET/POST/PATCH /v1/transactions, GET /:id	M3
ingestion	POST /v1/ingestion/events, GET event status for debug	M4
review	GET /v1/reviews, POST /:id/resolve	M6-M7
merchants/rules	GET/PATCH merchants, CRUD rules	M6
budgets	CRUD /v1/budgets, GET status	M8
integrations	Gmail connect/status/sync endpoints + cron trigger	M9
statements	upload/import/status/reconciliation endpoints	M10
reports	weekly/monthly summary endpoints + scheduled generation	M11

Pagination. Use cursor pagination for transaction/review feeds. The architecture guide explicitly recommends cursor/keyset pagination for mutable feeds and offset only where numbered pages are a real requirement (pages 34-37).

OpenAPI. Treat the API specification as the cross-repository contract. The guide derives OpenAPI from Zod and Drizzle schemas (pages 38-41); this is more important, not less, when Flutter and Nuxt live in separate repositories.

9. Development Roadmap

Effort is expressed in engineering days, not calendar dates. It assumes one developer and includes implementation + tests + documentation, but not prolonged provider-specific reverse engineering.

ID	Milestone	Effort	Main deliverables	Exit criteria
M0	Decisions + repo bootstrap	1-2	API repo, Bun/npm choice, lint/type rules, environments, ADRs, initial README.	Repository builds and deploys hello/health Worker.
M1	Foundation	2-3	Hono app factory, env validation, container, logger, errors, middleware, OpenAPI, CI.	Health/OpenAPI deploy; request IDs and error envelope verified.
M2	Auth + tenancy seam	2-3	Better Auth, bearer plugin, principal, signup policy, auth middleware, user ownership conventions.	Flutter-capable bearer auth works; protected endpoint contract tests pass.
M3	Ledger core	4-5	Accounts, transactions, categories, merchants base schema; manual transaction API.	Can create/list/update a user-scoped transaction reliably.
M4	Raw ingestion	4-5	raw_financial_events, Android notification payload contract, idempotency, processing state/retry.	Repeated same event creates one raw event and never double-spends.
M5	Parsing + matching	4-6	Provider parser interface, normalization, account resolution, transaction_sources, dedup/matching.	Two observations of same purchase converge to one transaction.
M6	Classification	4-6	Merchant aliases, deterministic rules, personal/business context, Workers AI fallback + validation.	Known merchants auto-classify; ambiguous events produce suggestions safely.
M7	Transfers + review queue	4-6	Internal transfer matcher, credit-card payments, cash withdrawal mapping, review resolution + learning.	Transfers excluded from spend; review resolution improves next occurrence.
M8	Budgets + alert records	3-4	Monthly/category budgets, threshold evaluation, alert/event records.	80/100/120% threshold logic is covered by tests.
M9	Gmail automation	3-5	OAuth connection, encrypted tokens, provider filters/parsers, scheduled polling, sync cursor.	New bank/e-wallet email becomes a raw event automatically.
M10	Statements + reconciliation	6-8	R2 storage, CSV first, provider PDF adapters later, statement rows, matching/reconciliation.	Statement can confirm existing entries and recover missed transactions.
M11	Reports	3-4	Weekly/monthly aggregates, category trends, personal/business summaries, scheduled report generation.	Report totals reconcile with ledger queries.
M12	Hardening + release gate	5-7	Load tests, failure recovery, retention, secrets/redaction audit, migration drills, preview envs, smoke tests.	Personal production release can run unattended for 2-4 weeks.

Estimated backend effort: Approximately 45-60 engineering days for the full backend roadmap. The first useful backend/mobile integration checkpoint is M4-M5; the first low-touch personal finance MVP is M7.

10. First Implementation Sequence - Start Here

Checkpoint A - Bootstrap and deploy

1. Create mhel-finance-api as its own repository.
2. Initialize Hono for Cloudflare Workers and configure TypeScript strictness from the architecture guide.
3. Add Zod environment validation and wrangler dev/preview/production environments.
4. Add structured logger, request ID, security headers, CORS, centralized errors.
5. Expose /health and /openapi.json.
6. Add GitHub Actions: typecheck, lint, tests, migration diff check, deploy, smoke test.

Checkpoint B - Database and identity

1. Create Neon project and Drizzle client using the HTTP driver first.
2. Add Better Auth schema and configuration; enable bearer support for Flutter.
3. Create the initial user; disable public signup in personal mode.
4. Establish owner/user scoping conventions before adding finance tables.
5. Create Neon preview branches in pull-request CI once the base workflow is stable.

Checkpoint C - First vertical finance slice

1. Create account module and schema.
2. Create transaction module with manual create/list/update.
3. Use amount_minor bigint, currency, account_id, type, status, transaction_at, merchant_raw, category_id, usage context.
4. Add cursor pagination and indexes that match the sort order.
5. Write service tests and repository integration tests before ingestion automation.

Checkpoint D - Android ingestion contract

1. Define one generic raw event contract instead of provider-specific HTTP endpoints.
2. Persist event before parsing; enforce database uniqueness for idempotency.
3. Return quickly and process with waitUntil.
4. Add retry Cron for pending/failed events.
5. Only after this is stable should the Flutter notification listener upload real events.

11. Neon Transaction Strategy

The uploaded guide calls the Neon driver choice the most consequential infrastructure decision in this stack (page 19). For this project, start with the simpler HTTP driver and design atomic writes around constraints plus batch transactions.

Start with neon-http when...	Switch to WebSocket/interactive transactions when...
Reads can happen before the write phase and final writes can be batched atomically.	A business operation must read, branch on data, write, re-read, and conditionally write inside one transaction.
Uniqueness constraints/upserts safely resolve races.	You cannot preserve invariants with constraints/upserts + atomic batch writes.
You want stateless per-request DB client construction.	A proven workflow needs interactive <code>db.transaction(async tx => ...)</code> .

Financial integrity rule: Prefer database constraints and idempotent algorithms over long interactive transactions. If an invariant cannot be made safe that way, move that operation to the Neon WebSocket driver rather than weakening correctness.

12. Matching, Deduplication, and Transfers

Deduplication evidence

Evidence	Example	Weight
Exact source ID	Same Gmail message ID / Android event ID	Definitive duplicate raw event
Account + amount + time window	Same card, PHP 1,249, within minutes	Strong candidate
Merchant similarity	UNIQLO PH vs UNIQLO SM MEGAMALL	Supporting evidence
Statement reference / last4	Provider ref or masked account match	Strong when available
Opposite account movement	-10,000 bank and +10,000 GCash	Transfer candidate

Transfer rules

- Internal bank/e-wallet transfers become linked account movements, not expenses.
- Credit-card payment is a transfer from bank account to card liability, not new spending.
- ATM withdrawal is bank -> cash account. Actual cash purchases are separate manual transactions.
- Only auto-link high-confidence transfer pairs. Ambiguous candidates become review items.

13. Gmail and Statement Automation

Gmail - personal phase

- Poll incrementally on a Cloudflare Cron schedule instead of adding Pub/Sub complexity immediately.
- Use narrow sender/subject filters and provider-specific parsers.
- Persist processed Gmail message IDs so reruns are idempotent.
- Encrypt OAuth refresh tokens before storage using Web Crypto and a Worker secret.
- Provider parser failures must leave the raw event available for replay/debugging.

Statements - correctness phase

- Prefer CSV whenever the provider offers it.
- Store original statement files in R2 and metadata/checksum in Neon.
- Parse deterministically first; do not make Workers AI the primary statement parser.
- Reconciliation can confirm existing transactions, correct posted data, and discover missed transactions.
- PDF support is provider-by-provider and should be added only after the corresponding statement format is tested against real samples.

14. Testing Strategy

Follow the guide's test pyramid (pages 42-43): heavy service unit tests, repository integration tests against Postgres, route/OpenAPI contract tests without a network server, and a thin deployed E2E layer.

Test level	Finance-specific focus	Examples
Unit - service	Business invariants and classification policies.	Transfer does not count as spend; rule precedence; review resolution learning.
Parser fixtures	Provider text -> normalized event.	GCash/BPI/etc notification examples and malformed variants.
Repository integration	Constraints, indexes, queries, migrations.	Duplicate event rejected; user isolation; cursor order stable.
Contract	Route validation + response schemas.	app.request() against OpenAPI route definitions.
AI adapter unit	Model call isolated behind fake.	Invalid category rejected; missing fields fall back to review.
AI eval - staging	Small curated corpus against real Workers AI.	Measure accepted/review/error rate; no hard deterministic assertions.
E2E	Deployed Worker and Neon.	Login -> create account -> ingest event -> observe transaction/review.

CI architecture guard: Add the guide's layer-contract check: services must not reference Hono Context, c.req, c.json, HTTPException, or database modules directly. Architecture drift should fail CI.

15. Deployment, Migrations, and Observability

CI/CD order

```
PR / push
-> install
-> typecheck
-> lint + architecture boundary checks
-> unit + parser + contract tests
-> verify generated migrations are committed
-> repository integration tests (Neon branch)
-> production: apply migrations
-> deploy Cloudflare Worker
-> smoke test /health + critical authenticated flow
```

Migration discipline. The guide requires generated/reviewed/committed migrations and explicitly rejects db:push for production. Use expand -> backfill -> migrate reads -> contract for breaking changes (pages 44-45).

Minimum observability

- requestId on every log line, error body, response header, and outbound provider call.
- Structured http.request logs with method, path, status, duration, principalId.
- Domain processing logs: event.received, event.processed, event.failed, transaction.matched, review.created.
- Slow DB query logging above a measured threshold.
- Explicit redaction for tokens, cookies, email content fragments, account identifiers, and secrets.
- A health endpoint that verifies Worker liveness; deeper DB checks can be separate to avoid turning temporary DB issues into edge routing failures.

Do not add OpenTelemetry yet. The uploaded guide correctly treats request ID correlation as sufficient for a single service. Introduce distributed tracing when the backend actually becomes multiple services (page 46).

16. Security and Privacy Requirements

Area	Requirement
Secrets	Worker secrets only; never commit database URLs, OAuth secrets, Better Auth secret, encryption keys.
OAuth tokens	Encrypt Gmail/provider refresh tokens before storing in Neon.
Logs	Never log Authorization, cookies, raw access/refresh tokens, passwords, full account numbers.
Raw events	Define retention; keep enough for debugging/reconciliation, not forever by default.
API responses	Return public DTOs parsed through Zod schemas; never raw DB rows with hidden columns.
Authorization	All finance queries scoped by user; resource ownership checks in services.
Idempotency	Use DB uniqueness for financial event identity; client retries must be safe.
Files	R2 objects private; access only through authenticated backend or short-lived signed mechanisms when needed.
AI	Minimize payloads; validate outputs; no direct model-to-ledger writes.
Backups	Neon backup/restore policy tested before relying on the system for primary financial history.

17. Scaling Path - Personal to Paid Product

Stage	Keep	Add only when needed
Personal	One API Worker, Neon, Better Auth, Cron, Workers AI, R2 later.	Nothing else.
Private beta	Same API architecture; user_id already everywhere.	Signup/invite flows, stronger rate limits, RLS evaluation, usage metrics.
Paid launch	Modular API still acceptable.	Billing/subscription/entitlements, audit/admin tools, support workflows.
Growth	Keep domain contracts and OpenAPI.	Cloudflare Queues, dedicated processing Worker/repo, read-model/report optimizations.
Large scale/compliance	Keep canonical ledger semantics.	Service extraction only for measured bottlenecks/ownership/compliance boundaries.

No premature microservices: Do not create account-service, transaction-service, budget-service, AI-service repositories now. They would add network failure modes, distributed transactions, versioning, tracing, and deployment coordination without solving a current problem.

18. Definition of Done for Backend v1

- API repo deploys independently to Cloudflare production from CI.
- Neon migrations are versioned, reviewed, and tested against preview branches.
- Better Auth protects all finance endpoints; Flutter bearer session flow is proven.
- Accounts and canonical transactions are user-scoped and queryable with cursor pagination.
- Android notification events are idempotently ingested and recover after processing failures.
- At least two real providers have fixture-tested parsers.
- Multiple observations of one purchase do not produce duplicate expense totals.
- Classification order is deterministic rules -> heuristic -> Workers AI -> review.
- Transfers/credit-card payments are excluded from expense totals.
- Review corrections create safe learned mappings.
- Budget threshold records and weekly/monthly summary queries are correct.
- Gmail polling is automated and provider tokens are encrypted.
- CSV statement reconciliation works for at least one real statement format.
- Raw data retention, log redaction, backups, and restore procedure are documented.
- System runs unattended for 2-4 weeks with review limited to genuine exceptions.

19. Immediate Backend Backlog

Priority	Issue / story	Acceptance
P0	Bootstrap Hono Worker repo	Local wrangler dev + production deploy + /health.
P0	Implement env/container/logger/error foundation	Invalid env fails clearly; requestId in response/logs.
P0	Set up Drizzle + Neon migrations	Generate/commit/migrate workflow; no prod push.
P0	Integrate Better Auth	Cookie + bearer session verified.
P0	Create account module	User-scoped CRUD with tests.
P0	Create transaction module	Manual ledger CRUD + cursor list + money invariants.
P0	Create raw ingestion module	Generic event endpoint, DB idempotency, processing state.
P1	Implement first provider parser	Fixture suite from real Android notification samples.
P1	Implement event -> transaction matching	Duplicate observations converge.
P1	Implement review queue	Low-confidence event is actionable and resolvable.
P1	Add deterministic categorization rules	Known merchant becomes zero-touch.
P1	Add Workers AI classifier adapter	Only invoked after deterministic path fails.
P1	Implement transfer matching	Same-owner account movement does not affect spend.
P2	Gmail polling integration	New financial email flows through same event pipeline.
P2	Budgets/reports/statements	Build after capture + matching quality is proven.

20. Architecture Basis and Notes

Primary architecture reference: Backend Architecture Specification - Hono / Drizzle / Neon / Better Auth / Zod / Cloudflare Workers, version 1.0 (uploaded project document).

- Pages 3-7: explicit dependencies, thin transport, vertical slices, schema as source of truth, service isolation.
- Pages 10-19: composition root, Neon/Drizzle database layer, repositories, services, transaction driver caveat.
- Pages 20-33: middleware order, logging, auth, authorization, error envelopes and security boundaries.
- Pages 34-41: cursor pagination and OpenAPI/Zod route definition strategy.
- Pages 42-43: testing pyramid and Hono contract testing.
- Pages 44-45: migration discipline, expand/contract, CI/CD, Neon preview branches.
- Page 46: request ID correlation, minimum observability, redaction.
- Page 47: package manifest and strict TypeScript recommendations.

Roadmap interpretation: Items that are specific to the expense-tracker domain - canonical event/transaction model, Android/Gmail ingestion, transfer handling, Workers AI policy, budgets, reconciliation, and the staged scale path - are project design recommendations derived from the requirements established in this conversation, while the structural Hono practices are grounded in the uploaded architecture guide.

NEXT EXECUTION TARGET: M0 -> M1 -> M2 -> M3 -> M4

Do not build budgets, reports, or statement parsing before raw ingestion and deduplication are trustworthy.